

[Course Home \(/learn/data-structures-optimizing-performance/home/welcome\)](#) > [Week 1 \(/learn/data-structures-optimizing-performance/home/...](#)

Assignment: How Easy to Read is Your Writing?

You have not submitted. You must earn 2/2 points to pass.

Deadline Pass this assignment by November 2, 12:59 AM PDT.

Instructions ([/learn/data-structures-optimizing-performance/programming/neATU/how-easy-to-read-is-your-writing](#))

My submission ([/learn/data-structures-optimizing-performance/programming/neATU/how-easy-to-read-is-your-writing/submission](#))

Discussions ([/learn/data-structures-optimizing-performance/programming/neATU/how-easy-to-read-is-your-writing/discussions](#))

Programming Assignment for Module 1: How Easy to Read is Your Writing?

In this first programming assignment, which is the first part of the project you will develop throughout this course, you will write code to analyze the reading level of a piece of text. The Flesch Readability Score (https://en.wikipedia.org/wiki/Flesch%E2%80%93Kincaid_readability_tests) is a measure of the reading complexity of a piece of text. Developed by author Rudolf Flesch, it is a measure that approximates how easy a piece of text is to read based on the number of sentences, words and syllables in that text. Higher scores indicate text that is simple to read, while lower scores indicate more complex text.

In this programming assignment you will write the back-end code for calculating Flesch readability scores, and then you will integrate this back-end capability with the front-end application we provide that you will be adding to throughout this course.

Getting Set Up

1. (If not already done) Download and set up the starter code

Before you begin this programming assignment, you need to download and set up the starter code. You should have already followed the instructions for setting up Java and Eclipse (<https://www.coursera.org/learn/data-structures-optimizing-performance/supplement/amdH/setting-up-java-eclipse-and-the-starter-code>). Make sure you have the most recent version of the starter code. Because this course is so new and the starter and grading code is relatively complex, we occasionally need to update the starter code with bug fixes. The zip file on the set up page is annotated with the date it was uploaded, so if there's a new version since you downloaded it, you should grab it now. You are welcome to use any IDE of your choice, but we provide the starter code as an Eclipse project, so you'll probably find it easiest to work in Eclipse.

Make sure you've followed these setup instructions even if you've previously worked with Eclipse, and even if you took the first course in our series, because you'll need to make sure you have some additional pieces installed and set up (e.g. Java 1.8 latest version, JUnit).

2. Verify that the frontend runs

Make sure the project you downloaded and set up for this course is open in eclipse.

As you saw in Mia's demo video, all of the functionality you will implement in this course will be integrated into a text editor application. You can already run this application, but it won't do much.

Run the text editor by running the MainApp class inside the application package. You can do this by expanding the application package in the package explorer and then selecting the MainApp.java file and running it.

You will see a text editor window open. You can type in the text window and load text using the "Load" button. None of the other buttons work yet (you can click them, but they don't do anything). But they will soon...

3. Orient yourself to the starter code

In this project, you will write a class to represent a text Document. The Document class will be capable of calculating and returning properties of the text, including its Flesch readability score. Before you begin coding, you should get comfortable with the class hierarchy (if you need a refresher on inheritance and class hierarchies in Java, check out the first course (<https://www.coursera.org/learn/object-oriented-java>) in our specialization).

Open the starter code for this assignment by expanding the document package in the Package Explorer window. There you will see several files, but the two that are relevant to this assignment are Document.java and BasicDocument.java. Open these by double-clicking on them.

Notice that Document is an abstract class. BasicDocument will implement the abstract methods in the Document class using the guidelines below described in the parts below.

Assignment and Submission Details

This assignment is divided into two parts. You will submit a separate file for each part, as described below.

Part 1: Implement the missing methods in BasicDocument.java

In this class, `countSyllables`, `countWords`, and `countSentences` each calculate the number of syllables, words, and sentences, respectively, each time the method is called. **It should do this by processing the entire text of the document each time the method is called. It should NOT store this information after it calculates it**, but rather just return it. This class is not supposed to be efficient.

To facilitate auto-grading, you must use the following definitions of what constitutes a syllable, word, and sentence and make sure you are consistent with our test cases/examples in the main method.

public int getNumSentences()

This method counts and returns the number of sentences in the document. A sentence is any sequence of characters ending in one or more end of line punctuation marks, or the last sequence of characters in a document, even if they don't end with an end of line punctuation mark. An end of line punctuation mark includes a period (.), an exclamation point (!) and a question mark (?). That's it. Notice that we provide a main method that gives you several examples/test cases.

public int getNumWords()

This method counts and returns the number of words in the document (the stored text). A word is any contiguous sequence of one or more alphabetical characters. We will completely ignore numbers when we count words, and you can assume you will not have any strings that combine numbers and letters. Again, check our examples in the main method.

public int getNumSyllables()

This method counts and returns the total number of syllables in the document (the stored text). To count the number of syllables in a word, you should use the rule that every contiguous sequence of one or more vowels, except for a lone "e" at the end of a word if the word has another vowel or set of contiguous vowels, makes up one syllable. You should consider y a vowel. Under these rules the words "the", "fly", "yes", "cave" and "double" all have 1 syllable, but "segue" has two syllables. Notice that this isn't exactly correct ("double" actually has 2 syllables), but it's close enough for our purposes. Here are some more examples with the number of syllables your method should return to help you: "contiguous" (3 syllables), "sleepy" (2 syllables), "obvious" (2 syllables), "toga" (2 syllables). Notice that our rules get a lot wrong, especially when you have more than 2 vowels in a row, but these are the rules we will test you against.

Testing your code:

We have provided a method named `testCase` in the `Document` class. This method takes a `Document` object (which can be a `BasicDocument` object...) as well as the expected number of syllables, words, and sentences in that document and tests to make sure each method is correct, printing messages accordingly. You should make use of this method and add your own tests in the main method we provide.

Hints for this part:

- Notice the helper method `getTokens` in the `Document` superclass. It takes a string which is a regular expression representing the regular expression of the "tokens" it's trying to find. Consider passing a different pattern for each method. If you get really stuck here there is a support video that can help.
- Consider making a helper method in the `Document` class (i.e. the superclass) that counts the number of syllables in a single word. This will come in useful in next week's assignment as well. We have a stub method already in `Document.java`. You should not need to create `countWords` or `countSentences` methods in `Document.java` because these methods will not be useful next week.

Making your code work with the text editor application (optional, but encouraged)

Currently, the text editor application is not set up to use the `BasicDocument` class (it uses `EfficientDocument`, which you will implement next week). So if you want your text editor application to calculate the Flesch score (after you finish part 2), you must change one line of code in the file `LaunchClass.java`, which is in the application package. In the method

```
public document.Document getDocument(String text)
```

change `"document.EfficientDocument"` to `"document.BasicDocument"`. Remember to change this back next week!

What and how to submit

When you have tested your code and are ready to submit this part, you should open and run the file `BasicDocumentGrader.java` which you can also find in the document package. When you run that file, it will produce two output files: `module1.part1.out` and `module1.part2.out` in the folder `grader_output`. Don't worry about `module1.part2.out` for now.

On your computer, make sure you can find the `grader_output` folder in your workspace under the `MOOCTextEditor` folder. Inside that folder you should see the two files that the `BasicDocumentGrader` program produced.

Upload the file `module1.part1.out` as your submission for part 1 and click submit to see your grading feedback! **DO NOT submit your `BasicDocument.java` or your `Document.java` file.** If there are any errors, fix them and then try submitting again. You can see exactly what test cases we used by looking at the code for the grader and checking the file the grader is reading from. Note that due to limitations of the Coursera platform, we are only able to give you feedback on one error at a time, so once you fix your first error, that might reveal another one.

Part 1 Bonus (purely optional, just for fun):

Our REGEX are pretty naive. For example, the word 7.5 causes real problems for our expressions as a "." usually denotes the end of a sentence. If you want to learn more about REGEX, you can improve upon our approach. But if you want to do this, put it in a separate class (like `ImprovedDocument`) and make sure you don't use it to produce your grading output.

Part 2: Implement the `getFleschScore` method in `Document.java`

Now fill in the method `getFleschScore()` in `Document.java` to calculate the Flesch Score for the text in the document. You should use the following formula, and make calls to the `getNumSyllables`, `getNumWords`, and `getNumSentences` you just implemented.

You should test your code by calculating the Flesch score by hand on some very basic documents and then calling your method from main to make sure it's giving the same output.

What and how to submit

To submit part 2, you simply need to re-run the BasicDocumentGrader program you ran for part 1 and then submit the file module1.part2.out (which you can find the grad_output folder) as your submission for part 2. **DO NOT submit your Document.java file.** If there are any errors, you can again find them, fix them and resubmit.

Part 3 (nothing to submit): Have fun with calculating the readability of text that you find

Now that you've implemented the Flesch score, your GUI interface will automatically include this functionality. Try running it again, and playing around with calculating the Flesch score of various documents, either that you have produced or that you find on the web. We'd love to see fun things you discover on the discussion board!

How to submit

When you're ready to submit, you can upload files for each part of the assignment on the "My submission" tab.

<

(/learn/data-structures-optimizing-performance/lecture/HbU1R/project-programming-assignment-walk-through)

>

(/learn/data-structures-optimizing-performance/exam/xTNnj/module-and-programming-assignment-quiz)

<

(/learn/data-structures-optimizing-performance/lecture/HbU1R/project-programming-assignment-walk-through)

>

(/learn/data-structures-optimizing-performance/exam/xTNnj/module-and-programming-assignment-quiz)